

VolumetricSynth Midterm Presentation

progress update

Team: Angelina Zhai; Canting Zhu; Griffin Barbieri; Lennon Seiders; Pratham Vadhulas
Spring 2026



Georgia Tech · College of Design

Center for
Music Technology

Introduction

Project Introduction

- VolumetricSynth is a JUCE-based plugin for real-time 3D timbre blending.
- It extends classic 2D vector synthesis to an 8-corner 3D interpolation model.
- Goal: improve expressive sound design while keeping control intuitive.
- This presentation: goals, ownership, progress, remaining work, and timeline to May 5.



Introduction

Project Introduction

- VolumetricSynth is a JUCE-based plugin for real-time 3D timbre blending.
- It extends classic 2D vector synthesis to an 8-corner 3D interpolation model.
- Goal: improve expressive sound design while keeping control intuitive.
- This presentation: goals, ownership, progress, remaining work, and timeline to May 5.



Introduction

Project Introduction

- VolumetricSynth is a JUCE-based plugin for real-time 3D timbre blending.
- It extends classic 2D vector synthesis to an 8-corner 3D interpolation model.
- Goal: improve expressive sound design while keeping control intuitive.
- This presentation: goals, ownership, progress, remaining work, and timeline to May 5.



Introduction

Project Introduction

- VolumetricSynth is a JUCE-based plugin for real-time 3D timbre blending.
- It extends classic 2D vector synthesis to an 8-corner 3D interpolation model.
- Goal: improve expressive sound design while keeping control intuitive.
- This presentation: goals, ownership, progress, remaining work, and timeline to May 5.



Goals and Scope

- **Goal 1:** Build stable core DSP blocks for a volumetric synthesis pipeline.
- **Goal 2:** Integrate modules into a playable plugin path (MIDI to audio out).
- **Goal 3:** Provide a clear UI/control layer for 3D mix behavior.
- **In scope:** oscillator/filter/envelope/mixer integration, plugin wiring, tests, and core UI state sharing.
- **Out of scope:** advanced FX suite, large preset ecosystem, and deep visual polish.

Architecture Overview

Section Roadmap

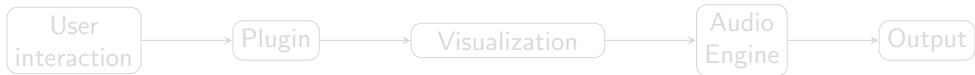
- Codebase structure overview.
- Views: Plugin, Visualization, Audio Engine.
- Stack: JUCE + OpenGL.
- Per-view diagrams
- Final frame: cross-module connections.



Architecture Overview

Section Roadmap

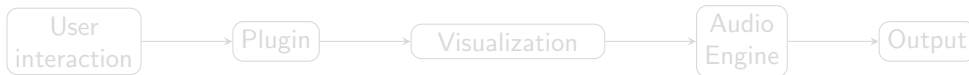
- Codebase structure overview.
- Views: Plugin, Visualization, Audio Engine.
- Stack: JUCE + OpenGL.
- Per-view diagrams
- Final frame: cross-module connections.



Architecture Overview

Section Roadmap

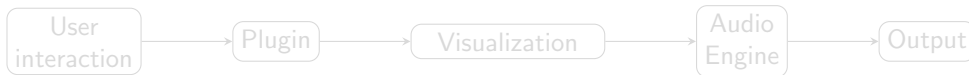
- Codebase structure overview.
- Views: Plugin, Visualization, Audio Engine.
- Stack: JUCE + OpenGL.
- Per-view diagrams
- Final frame: cross-module connections.



Architecture Overview

Section Roadmap

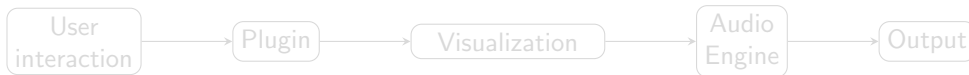
- Codebase structure overview.
- Views: Plugin, Visualization, Audio Engine.
- Stack: JUCE + OpenGL.
- Per-view diagrams
- Final frame: cross-module connections.



Architecture Overview

Section Roadmap

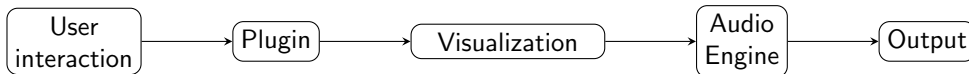
- Codebase structure overview.
- Views: Plugin, Visualization, Audio Engine.
- Stack: JUCE + OpenGL.
- Per-view diagrams
- Final frame: cross-module connections.



Architecture Overview

Section Roadmap

- Codebase structure overview.
- Views: Plugin, Visualization, Audio Engine.
- Stack: JUCE + OpenGL.
- Per-view diagrams
- Final frame: cross-module connections.



Architecture Overview

Plugin

- Host-facing plugin core and lifecycle.
- Main editor and UI composition root.
- Centralized parameter model (APVTS).
- **Flow:** host → processor → parameter/state bridge → audio + UI sync.

Processor Class Card

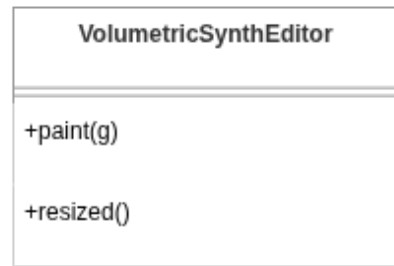
VolumetricSynthAudioProcessor
+prepareToPlay(sampleRate, samplesPerBlock)
+processBlock(audioBuffer, midiBuffer)
+createEditor()
+parameterChanged(parameterID, newValue)

Architecture Overview

Plugin

- Host-facing plugin core and lifecycle.
- Main editor and UI composition root.
- Centralized parameter model (APVTS).
- **Flow:** host → processor → parameter/state bridge → audio + UI sync.

Editor Class Card

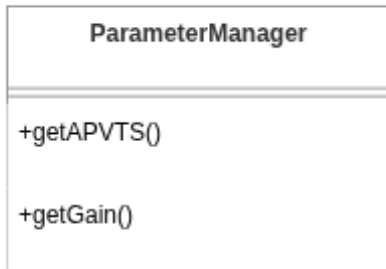


Architecture Overview

Plugin

- Host-facing plugin core and lifecycle.
- Main editor and UI composition root.
- Centralized parameter model (APVTS).
- **Flow:** host → processor → parameter/state bridge → audio + UI sync.

Parameter Manager Class Card

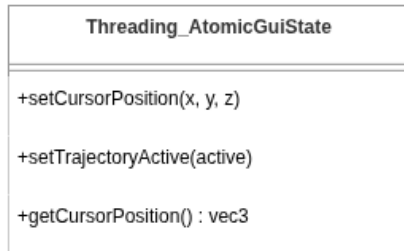


Architecture Overview

Plugin

- Host-facing plugin core and lifecycle.
- Main editor and UI composition root.
- Centralized parameter model (APVTS).
- **Flow:** host → processor → parameter/state bridge → audio + UI sync.

Atomic GUI State Class Card



Architecture Overview

Visualization

- OpenGL cube view renders the 3D interaction space.
- Mouse drag drives cube rotation and cursor updates.
- Cursor position is normalized and pushed through a lock-free state bridge.
- **Flow:** user gesture → 3D view update → cursor state → DSP.

3D View Class Card

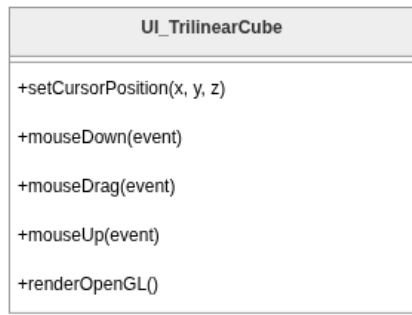
UI_TrilinearCube
+setCursorPosition(x, y, z)
+mouseDown(event)
+mouseDrag(event)
+mouseUp(event)
+renderOpenGL()

Architecture Overview

Visualization

- OpenGL cube view renders the 3D interaction space.
- Mouse drag drives cube rotation and cursor updates.
- Cursor position is normalized and pushed through a lock-free state bridge.
- **Flow:** user gesture → 3D view update → cursor state → DSP.

Interaction Loop Class Card



Architecture Overview

Visualization

- OpenGL cube view renders the 3D interaction space.
- Mouse drag drives cube rotation and cursor updates.
- Cursor position is normalized and pushed through a lock-free state bridge.
- **Flow:** user gesture → 3D view update → cursor state → DSP.

Cursor State Class Card

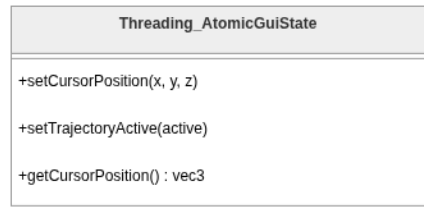
UI_CenterControlPanel
+setCursorChangedCallback(callback)
+setCursorPosition(x, y, z)
+paint(g)

Architecture Overview

Visualization

- OpenGL cube view renders the 3D interaction space.
- Mouse drag drives cube rotation and cursor updates.
- Cursor position is normalized and pushed through a lock-free state bridge.
- **Flow:** user gesture → 3D view update → cursor state → DSP.

Thread Bridge Class Card



Architecture Overview

Audio Engine

- MIDI and audio block orchestration.
- Voice allocation and note lifecycle.
- Per-voice synthesis signal path.
- **Flow:** MIDI/input → voice path → oscillator/mixer/envelope → output.

Synth Engine Class Card

Audio_SynthEngine
+prepare(sampleRate, blockSize)
+processBlock(audioBuffer, midiBuffer)
+noteOn(channel, note, velocity)
+noteOff(channel, note, velocity)

Architecture Overview

Audio Engine

- MIDI and audio block orchestration.
- Voice allocation and note lifecycle.
- Per-voice synthesis signal path.
- **Flow:** MIDI/input → voice path → oscillator/mixer/envelope → output.

Voice Manager Class Card

Audio_VoiceManager
<pre>+prepare(sampleRate, blockSize) +noteOn(channel, note, velocity) +noteOff(channel, note, velocity) +renderNextBlock(outputBuffer, startSample, numSamples)</pre>

Architecture Overview

Audio Engine

- MIDI and audio block orchestration.
- Voice allocation and note lifecycle.
- Per-voice synthesis signal path.
- **Flow:** MIDI/input → voice path → oscillator/mixer/envelope → output.

Synth Voice Class Card

Audio_SynthVoice
+prepare(sampleRate, blockSize)
+startNote(note, velocity, sound, pitchWheel)
+stopNote(velocity, allowTailOff)
+renderNextBlock(outputBuffer, startSample, numSamples)

Architecture Overview

Audio Engine

- MIDI and audio block orchestration.
- Voice allocation and note lifecycle.
- Per-voice synthesis signal path.
- **Flow:** MIDI/input → voice path → oscillator/mixer/envelope → output.

DSP Stack Class Card

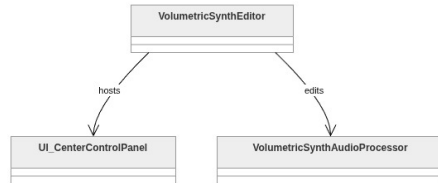
DSP_TrilinearMixer8
+prepare(sampleRate)
+processBlock(input, output, channels, samples)
+updateGainsFromPosition(x, y, z)

Architecture Overview

End-to-End Integration

- User interaction enters through the editor layer.
- Control and state propagate into real-time paths.
- Rendering path flows from engine to voices to DSP output.
- **Flow:** UI → control/state bridge → processor → audio output.

System Integration Overview

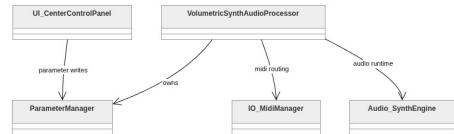


Architecture Overview

End-to-End Integration

- User interaction enters through the editor layer.
- Control and state propagate into real-time paths.
- Rendering path flows from engine to voices to DSP output.
- **Flow:** UI → control/state bridge → processor → audio output.

Control Path Focus

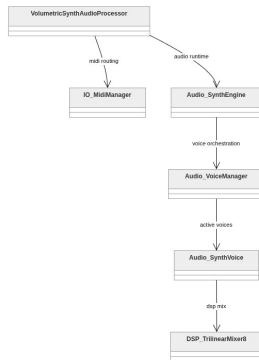


Architecture Overview

End-to-End Integration

- User interaction enters through the editor layer.
- Control and state propagate into real-time paths.
- Rendering path flows from engine to voices to DSP output.
- **Flow:** UI → control/state bridge → processor → audio output.

Processor Coordination Focus

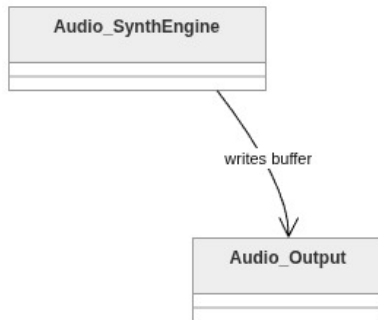


Architecture Overview

End-to-End Integration

- User interaction enters through the editor layer.
- Control and state propagate into real-time paths.
- Rendering path flows from engine to voices to DSP output.
- **Flow:** UI → control/state bridge → processor → audio output.

Render Pipeline Focus



Individual Responsibilities

- **Architecture/Integration (Role A):** plugin skeleton, module boundaries, build integration, and validation/QA ownership.
- **Audio Engine (Role B):** voice management, routing, and end-to-end realtime-safe orchestration.
- **DSP / Audio Effects (Role C):** DSP building blocks (e.g., mixer, filters), algorithm implementation and tuning.
- **UI + State Handoff (Role D):** editor controls, gain visualization, and parameter linking.

Individual Responsibilities

- **Architecture/Integration (Role A):** plugin skeleton, module boundaries, build integration, and validation/QA ownership.
- **Audio Engine (Role B):** voice management, routing, and end-to-end realtime-safe orchestration.
- **DSP / Audio Effects (Role C):** DSP building blocks (e.g., mixer, filters), algorithm implementation and tuning.
- **UI + State Handoff (Role D):** editor controls, gain visualization, and parameter linking.

Individual Responsibilities

- **Architecture/Integration (Role A):** plugin skeleton, module boundaries, build integration, and validation/QA ownership.
- **Audio Engine (Role B):** voice management, routing, and end-to-end realtime-safe orchestration.
- **DSP / Audio Effects (Role C):** DSP building blocks (e.g., mixer, filters), algorithm implementation and tuning.
- **UI + State Handoff (Role D):** editor controls, gain visualization, and parameter linking.

Individual Responsibilities

- **Architecture/Integration (Role A):** plugin skeleton, module boundaries, build integration, and validation/QA ownership.
- **Audio Engine (Role B):** voice management, routing, and end-to-end realtime-safe orchestration.
- **DSP / Audio Effects (Role C):** DSP building blocks (e.g., mixer, filters), algorithm implementation and tuning.
- **UI + State Handoff (Role D):** editor controls, gain visualization, and parameter linking.

Progress So Far

Since Jan 20

- Core repository and CMake structure established.
- Foundational DSP modules implemented (DualFilter, TrilinearMixer8, math helpers).
- Plugin processor/editor path in place for iterative integration.
- Unit tests added for critical DSP/math behavior.
- Current status: strong module-level progress; full end-to-end polish still in progress.

Progress So Far

Since Jan 20

- Core repository and CMake structure established.
- Foundational DSP modules implemented (DualFilter, TrilinearMixer8, math helpers).
- Plugin processor/editor path in place for iterative integration.
- Unit tests added for critical DSP/math behavior.
- Current status: strong module-level progress; full end-to-end polish still in progress.

Progress So Far

Since Jan 20

- Core repository and CMake structure established.
- Foundational DSP modules implemented (DualFilter, TrilinearMixer8, math helpers).
- Plugin processor/editor path in place for iterative integration.
- Unit tests added for critical DSP/math behavior.
- Current status: strong module-level progress; full end-to-end polish still in progress.

Progress So Far

Since Jan 20

- Core repository and CMake structure established.
- Foundational DSP modules implemented (DualFilter, TrilinearMixer8, math helpers).
- Plugin processor/editor path in place for iterative integration.
- Unit tests added for critical DSP/math behavior.
- Current status: strong module-level progress; full end-to-end polish still in progress.

Progress So Far

Since Jan 20

- Core repository and CMake structure established.
- Foundational DSP modules implemented (DualFilter, TrilinearMixer8, math helpers).
- Plugin processor/editor path in place for iterative integration.
- Unit tests added for critical DSP/math behavior.
- Current status: strong module-level progress; full end-to-end polish still in progress.

Remaining Tasks

- Finish full voice/engine integration path and verify stability under load.
- Tighten parameter mapping and edge-case behavior for UI and DSP interactions.
- Expand testing for integration-level scenarios (not only unit modules).
- Finalize end-to-end polish pass before submission.

Remaining Tasks

- Finish full voice/engine integration path and verify stability under load.
- Tighten parameter mapping and edge-case behavior for UI and DSP interactions.
- Expand testing for integration-level scenarios (not only unit modules).
- Finalize end-to-end polish pass before submission.

Remaining Tasks

- Finish full voice/engine integration path and verify stability under load.
- Tighten parameter mapping and edge-case behavior for UI and DSP interactions.
- Expand testing for integration-level scenarios (not only unit modules).
- Finalize end-to-end polish pass before submission.

Remaining Tasks

- Finish full voice/engine integration path and verify stability under load.
- Tighten parameter mapping and edge-case behavior for UI and DSP interactions.
- Expand testing for integration-level scenarios (not only unit modules).
- Finalize end-to-end polish pass before submission.

Comprehensive Timeline (Jan 20 to May 5)

Time	Focus	Owners	Done
Jan 20–Feb 2	Kickoff + architecture	A + D	Build baseline OK
Feb 3–Feb 23	DSP core modules	B	Module tests pass
Feb 24–Mar 16	Plugin integration	A + C	Audio path works
Mar 17–Mar 31	UI + state + tests	C + D	UI updates reliably
Apr 1–Apr 14	Feature completion (1)	B + C	Core features done
Apr 15–Apr 24	Optimization and Cleanup (2)	All	Stable integrated build
Apr 25–May 1	Demo + docs + rehearsal	All	Script + slides ready
May 2–May 5	Buffer + polish + submit	All	Submission package delivered

Demo

- video demo
- thanks, questions?

Demo

- video demo
- thanks, questions?